# CSS3 |Layout | flexbox

# CSS3 | Layout | flexbox

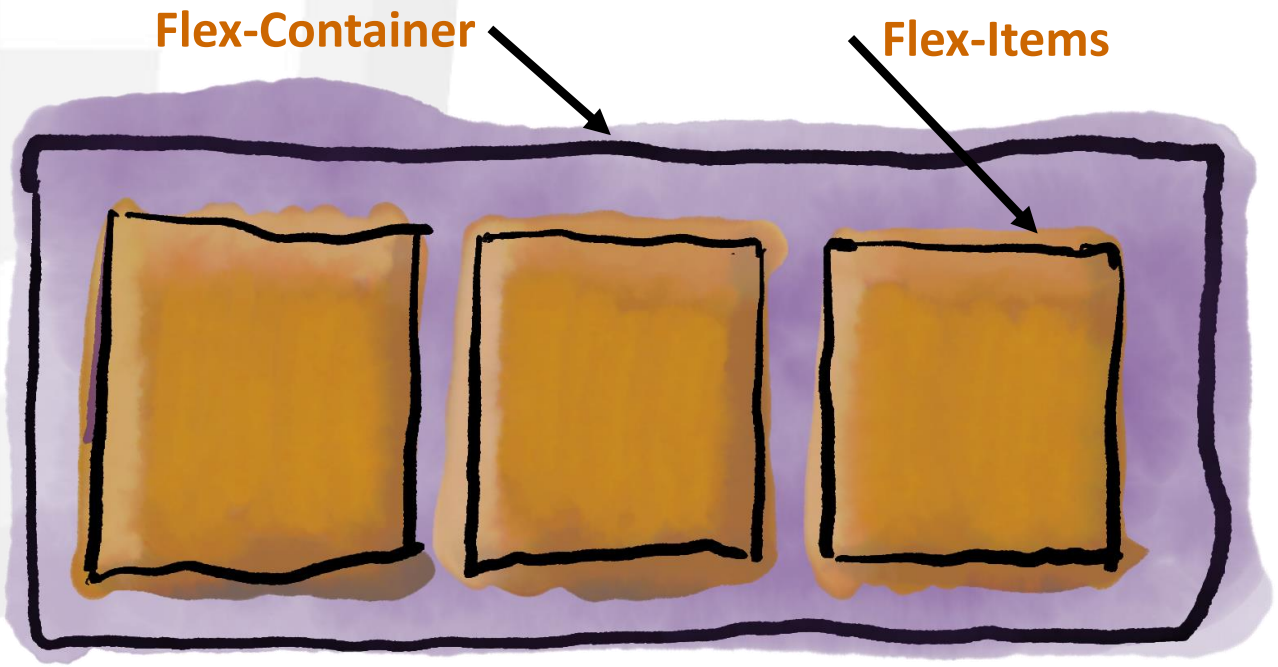
# CSS3 | Layout | flexbox

## CSS-Positionierung | Die neuen flexiblen Boxen von CSS3

Da das Thema Responsive Webdesign ist aus dem modernen Webdesign nicht mehr wegzudenken, deshalb wurde mit der Einführung von CSS3 der Wunsch nach einer einfacheren und besseren Alternative zur Positionierung immer lauter. **Diesem dringenden Wunsch der Community wurde mit dem Flexbox Modul für CSS3 entsprochen** versehen wurde, wird auch als **Container-Element** (= Behälter-Element) bezeichnet

Das Prinzip von Flexboxen ist relativ einfach und schnell erklärt.
**Sie benötigen ein Eltern-Element, bei dem Sie die CSS-Eigenschaft display auf flex setzen.**
**Diese Eigenschaft wirkt sich nun auf alle darin enthaltenen Elemente aus.**

Das Element, das mit der CSS-Eigenschaft **display: flex;** versehen wurde, wird auch als Container-Element (= Behälter-Element) bezeichnet.



# CSS3 | Layout | flexbox

**Flexbox wurde explizit zum Erstellen von Layouts geschaffen**, und das merkt man von Anfang an.
In einer Flexbox:

- **stehen alle Elemente automatisch nebeneinander.**
- **sind alle Elemente automatisch gleich hoch.**
- **werden alle darin enthaltenen Elemente umschlossen.**

Falls Sie jemals versucht haben, ein Float-Layout zu erstellen: Das Spiel hat noch nicht mal wirklich angefangen, und schon steht es 3:0 für Flexbox. Aber es geht noch weiter. Mit Flexbox können Sie unter anderem:

- **ganz einfach die Fließrichtung der Boxen ändern.**
- **verfügbaren Leerraum unterschiedlich verteilen.**
- **die Abmessungen der Boxen vom Browser berechnen lassen.**
- **Boxen ganz einfach horizontal und vertikal zentrieren.**
- **die Reihenfolge der Boxen am Bildschirm ändern.**

Die **Stärken** von **Flexbox** liegen in der Gestaltung von Modulen und Komponenten, in denen HTML-Elemente in einer Zeile oder in einer Spalte angeordnet sind, wie zum Beispiel bei der Ausrichtung von Links in einer horizontalen Navigation.

# CSS3 | Layout | flexbox

## Can I use

https://caniuse.com/#search=flexbox

Can I use flexbox ? Settings

3 results found

CSS Flexible Box Layout Module - CR

Method of positioning elements in horizontal or vertical stacks. Support includes all properties prefixed with `flex`, as well as `display: flex`, `display: inline-flex`, `align-content`, `align-items`, `align-self`, `justify-content` and `order`.

Usage % of all users ?

Global 96.49% + 1.29% = 97.78%

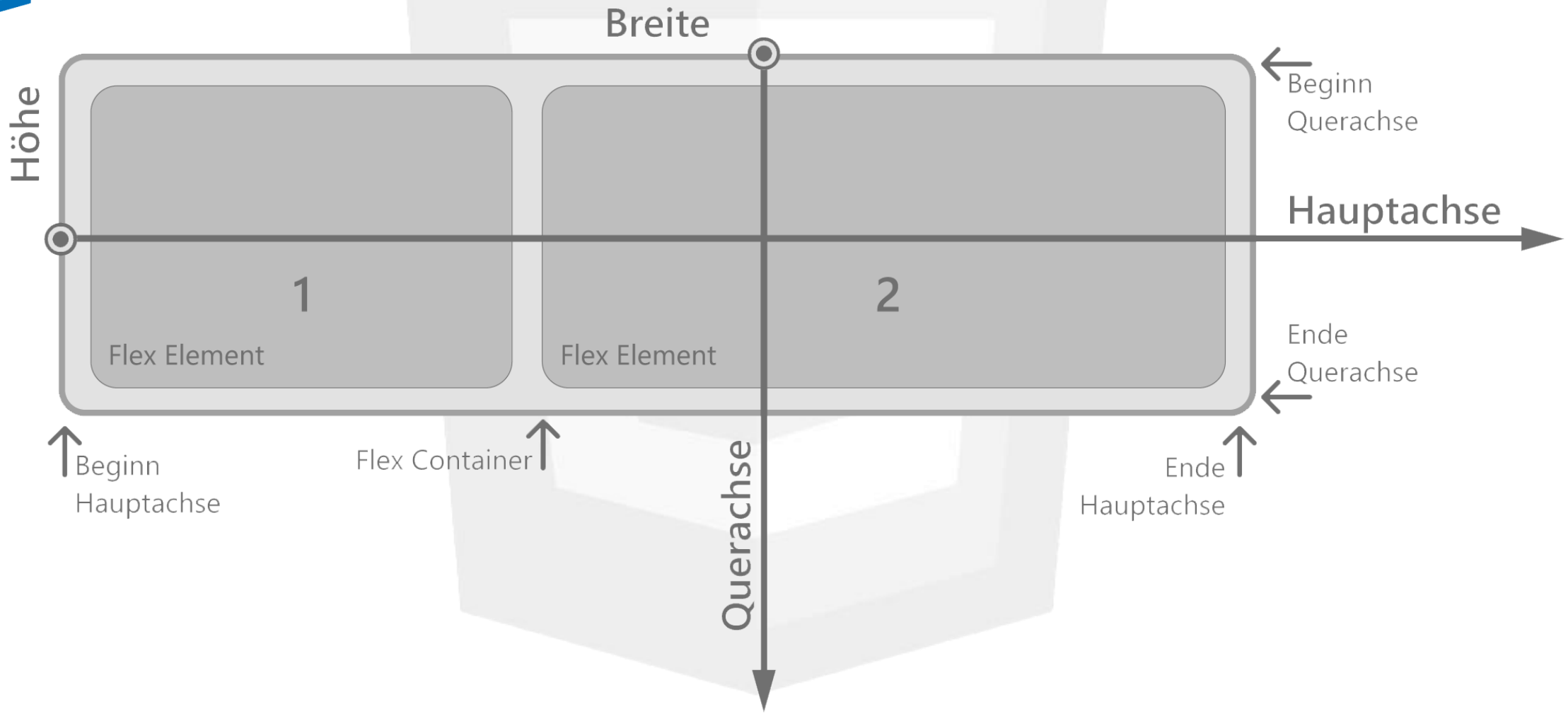
unprefixed: 96.4% + 0.89% = 97.29%

Current aligned | Usage relative | Date relative | Filtered | All

| IE | Edge * | Firefox | Chrome | Safari | Opera | Safari on iOS * | Opera Mini * | Android Browser * | Opera Mobile * | Chrome for Android | Firefox for Android | UC Browser for Android | Samsung Internet | QQ Browser | Baidu Browser | KaiOS Browser |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | 10-11.5 | | | | | | | | | | | |
| | | 1 2-21 | 1 4-20 | 1 3.1-6 | 12.1 | 1 3.2-6.1 | | | | | | | | | | |
| 6-9 | | 3 22-27 | 21-28 | 6.1-8 | 15-16 | 7-8.4 | | 1 2.1-4.3 | 12 | | | | | | | |
| 2,4 10 | 12-101 | 28-100 | 29-101 | 9-15.4 | 17-85 | 9-15.4 | | 4.4-4.4.4 | 12.1 | | | | 4-15.0 | | | |
| 4 11 | 102 | 101 | 102 | 15.5 | 86 | 15.5 | all | 101 | 64 | 102 | 101 | 12.12 | 16.0 | 10.4 | 7.12 | 2.5 |
| | | 102-103 | 103-105 | 16.0-TP | 87 | 16.0 | | | | | | | | | | |

Notes | Test on a real browser | Sub-features | Known issues (9) | Resources (12) | Feedback

# CSS3 | Layout | flexbox | Aufbau



# CSS3 | Layout | flexbox | Aufbau

**Die Flexbox ausrichten**

Wie Sie die Elemente innerhalb der Flexbox ausrichten, geben Sie mit der CSS-Eigenschaft flex-direction an.

Für eine horizontale Ausrichtung können Sie den Wert **row** (als Zeile) und für eine vertikale Ausrichtung den Wert **column** (als Spalte) verwenden.

```
section{
    width: 70%;
    height: auto;
    border: 2px solid black;
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row; /*Standardwert*/
```

**Verwenden Sie die CSS-Eigenschaft flex-direction nicht, dann ist row die Standardeinstellung.**
**Die Standardausrichtung für die Elemente einer Flexbox ist somit horizontal.**

**Das hier verwendete section Element muss natürlich nicht die section sein. Es kann jedes HTML Element sein, dass andere Elemente einschließen kann.**

# CSS3 | Layout | flexbox | Aufbau

**Eine Flexbox definieren mit display: flex**

**Der erste Schritt auf dem Weg zum Gestalten per Flexbox ist es immer, ein Element mit display: flex zu einem Flex-Container zu machen.** Dadurch werden alle Kindelemente automatisch zu Flex-Items.

Die Flexbox basiert auf einer strikten Eltern-Kind-Beziehung:

- **Das umgebende Elternelement ist der Flex-Container.**
- **Die darin enthaltenen Kindelemente werden zu Flex-Items.**

**Nach außen verhält sich ein Flex-Container wie eine Blockbox. Nach innen ist er eine Flexbox, in der neue Eigenschaften und Regeln wirken können**.

Vor dem Gestalten per Flexbox steht die Entscheidung, welches Element sich als Flex-Container eignet. In einer Navigationsleiste zum Beispiel möchten Sie die Listenelemente nebeneinanderstellen, und **da Flexbox auf dem Eltern-Kind-Prinzip basiert, wird die Liste ul zum Flex-Container, nicht das umgebende nav-Element**:

```
nav ul{
    display: flex;
}
```

# CSS3 | Layout | flexbox

## Artikel 1

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Reprehenderit, ea a quisquam, impedit libero pariatur amet veniam eaque est facilis aspernatur maiores repudiandae beatae! Doloribus magni obcaecati, eveniet nisi. Inventore dolorum ea quia veniam sapiente odio nam aliquid, labore ex, quis soluta debitis ad, excepturi est magnam magni eaque, iure.

## Artikel 2

Lorem ipsum dolor sit amet, consectetur adipisicing elit. Maiores fugit et saepe recusandae aliquid minima dolores nisi ratione beatae temporibus explicabo nobis, asperiores soluta sapiente. Cumque porro iure unde accusantium magni, at laboriosam perspiciatis alias in nostrum iste laudantium soluta voluptates! Animi sed itaque aut tenetur, illo quas minima molestias.

## Artikel 3

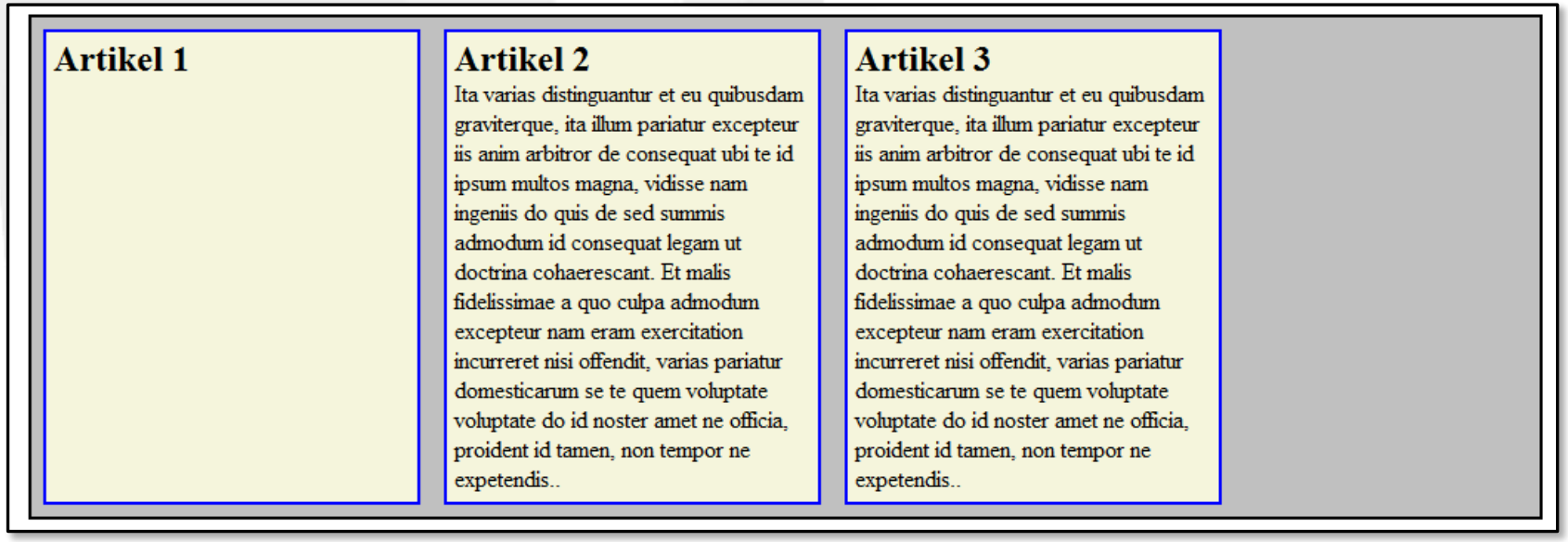
Lorem ipsum dolor sit amet, consectetur adipisicing elit. Repellat possimus dignissimos libero necessitatibus nisi debitis consequuntur saepe doloremque sint dolorum pariatur cumque, atque ratione inventore suscipit, quia, ad aliquam hic sequi! Nisi repellat velit quo corporis quam vel quaerat natus consequatur aliquam cumque deserunt officia, culpa, eius expedita cum maiores?

**flex-direction: row;**

# CSS3 | Layout | flexbox

Das Ergebnis sieht auf den ersten Blick aus wie float:left, doch im Detail gibt es bereits jetzt kleine, aber feine Unterschiede:

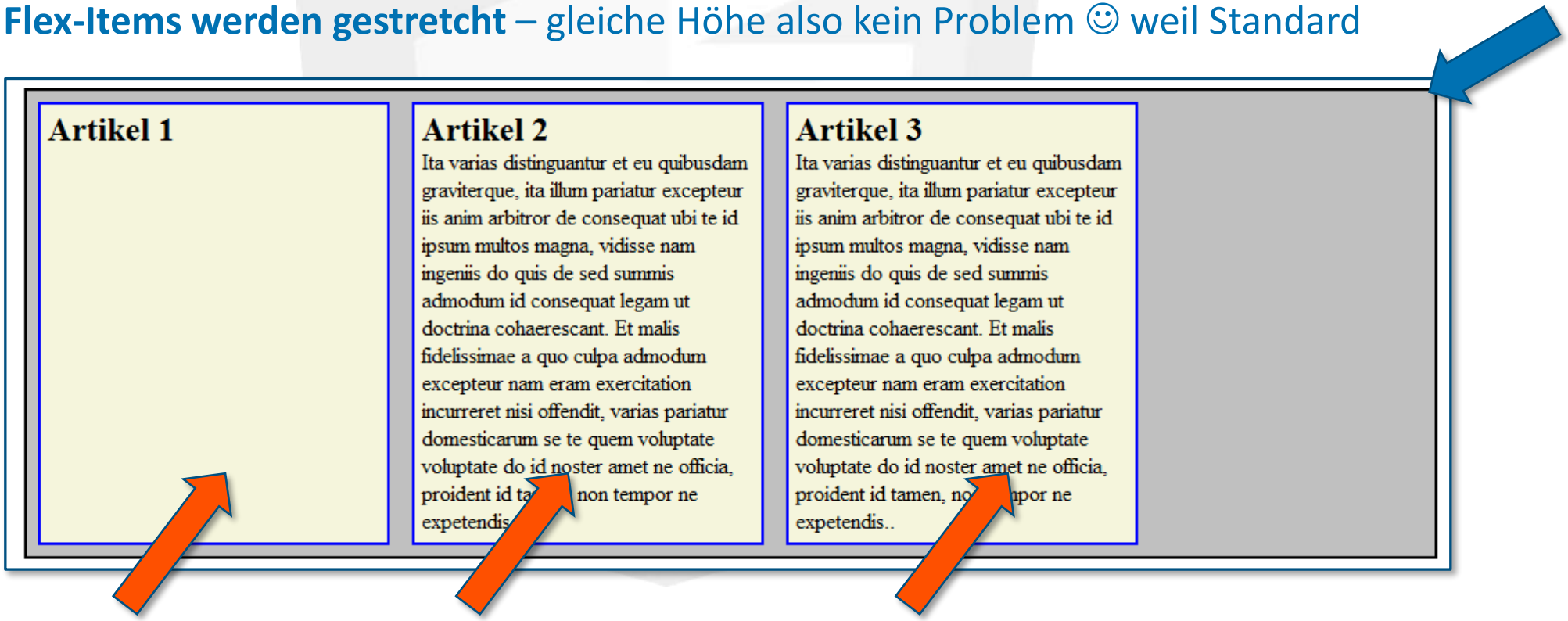
- **Der umgebende Flex-Container kollabiert nicht**, sondern umschließt die darin enthaltenen Kindelemente weiterhin.
- Ein clear ist nicht nötig, da eine **Flexbox ein in sich geschlossenes System** ist und vorherige oder nachfolgende Elemente nicht beeinflusst.
- **Die Höhe von Flex-Items ist unabhängig von ihrem Inhalt**. Flex-Items sind standardmäßig immer so hoch wie der umgebende Flex-Container.



# CSS3 | Layout | flexbox

**Eine Flexbox besteht aus Flex-Container und Flex-Items**

- Das umgebende Element mit **display:flex** ist ein **Flex-Container**.
- Die darin enthaltenen **Kindelemente** samt enthaltenen Content nennt man **Flex-Items**.
- **Die Flex-Items werden gestretcht** – gleiche Höhe also kein Problem ☺ weil Standard



# CSS3 | flexbox | flex-direction

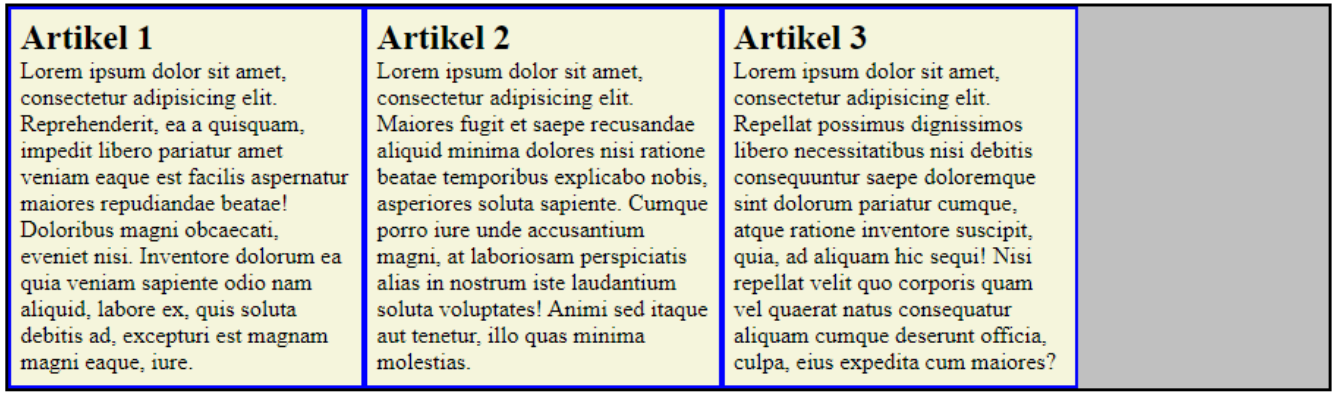
# CSS3 | Layout | flexbox

Hier wurde lediglich mit section das Eltern-Element die Anzeigeart **display** auf **flex** gesetzt und die Ausrichtung von **flex-direction** auf **row** (Standard).

Diese Angabe mit flex-direction können Sie aber auch ganz weglassen, weil **display: flex;** von Haus aus auf diese Einstellung eingerichtet ist.

Durch diese beiden Angaben werden alle in der section enthaltenen Kind-Elemente (hier: <article>-Elemente) entsprechend in der Flexbox (die hier zur Verdeutlichung in Grau eingefärbt wurden) horizontal ausgerichtet, wie Sie im Screenshot sehen.

```
section{
    width:70%;
    height: auto;
    border: 2px solid black;
    background: silver;
    margin: 2rem auto;
    display: flex;
```



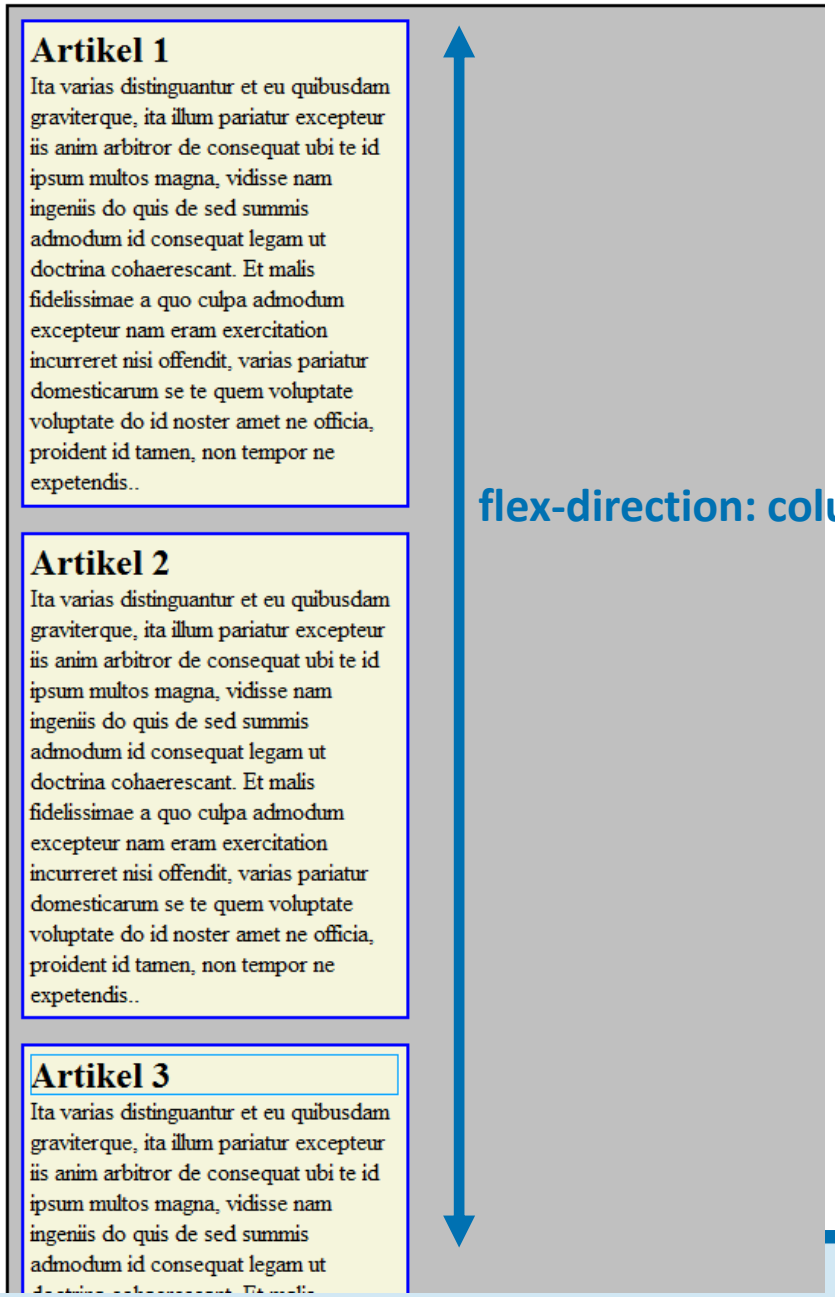
# CSS3 | Layout | flexbox

**flex-direction ändert die Fließrichtung: von row zu column**

**Mit der Eigenschaft flex-direction können Sie die Fließrichtung der Hauptachse und somit die Anordnung der Flex-Items ändern:**

- **flex-direction: row** ist der Standardwert, und die Flex-Items stehen nebeneinander in einer Zeile von links nach rechts.
- **flex-direction: column** dreht die Box um 90° auf die Seite, und die Flex-Items stehen dann untereinander in einer Spalte.

**row-reverse und column-reverse kehren die Fließrichtung um**
Äquivalent zu row und column gibt es die Werte row-reverse und column-reverse, die die Fließrichtung der Hauptachse um 180° ändern und die Reihenfolge der Flex-Items umkehren.




**flex-direction: column;**

# CSS3 | Layout | flexbox

## Völlig anders als bei float ist, dass die Flexbox bei einem kleineren Viewport nicht umbricht.

```
section{
    width:70%;
    height: auto;
    border: 2px solid black;
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row;/*Standardwert*/
```

```
article{
    border: 2px solid blue;
    padding: 0.5rem;
    width: 25%;
    height: auto;
}
```

### Artikel 1

Lorem ipsum dolor sit amet consectetur adipisicing elit. Sint quod doloribus, a enim beatae dolore tempore animi. Sint quisquam iste sed facilis ad tenetur reprehenderit? Nisi, voluptas! Doloremque in labore saepe, pariatur aliquam numquam possimus? Incidunt facere dolorum quas rem eaque ducimus harum neque earum voluptate aliquid dignissimos, officiis ut debitis vitae mollitia voluptatem corporis suscipit sint vel facilis quidem aut similique. Voluptas unde, voluptates incidunt at exercitationem earum, aut labore blanditiis nesciunt architecto cupiditate, repellat delectus adipisci possimus provident mollitia! Tenetur iure eius a alias dignissimos voluptatum cum officia culpa, vero delectus temporibus porro doloremque facilis minus earum perferendis.

### Artikel 2

Lorem ipsum, dolor sit amet consectetur adipisicing elit. Ducimus nobis animi labore quidem veritatis esse eos sint! Earum omnis consequatur nemo facilis inventore voluptatem itaque voluptates nihil doloribus accusantium ratione modi quasi, autem id atque rem veritatis sint assumenda provident cupiditate tempore impedit minima laboriosam cumque. Laboriosam dignissimos laudantium ea ex, quae vero iure harum provident atque reprehenderit architecto? Suscipit sapiente perspiciatis, non exercitationem assumenda illum quis iusto esse eaque voluptate consequuntur deleniti, dolores provident itaque nisi doloribus saepe magnam! Hic quam labore illum cupiditate sunt recusandae dolorum, blanditiis nisi ad, quas, ipsum earum quod sequi. Illum nihil voluptas reprehenderit.

### Artikel 3

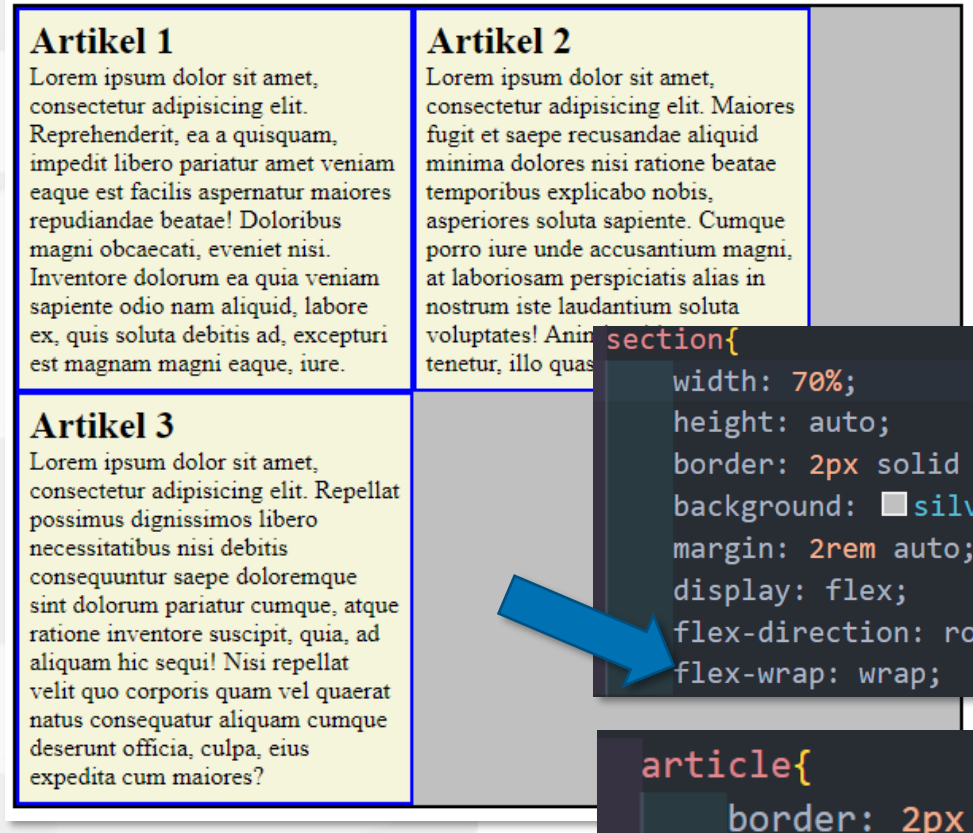
Lorem, ipsum dolor sit amet consectetur adipisicing elit. Aut corporis recusandae eligendi unde ex officiis explicabo soluta aperiam ipsam similique culpa, ad saepe facere consequuntur laudantium? Autem sed enim excepturi aliquam, odio ea rerum praesentium distinctio necessitatibus voluptatem alias atque et soluta natus hic nostrum ducimus minima blanditiis, rem laudantium ad, placeat laborum tempore! Iure sapiente ut sunt aliquam autem rerum ullam debitis ad quia cumque consectetur, quaerat minima ipsa quasi consequatur obcaecati quo placeat magnam. Doloremque, animi. Doloribus esse maxime beatae accusantium voluptas, velit nemo at nulla assumenda illum, incidunt explicabo accusamus odit voluptatibus enim pariatur aspernatur ad odio!

# CSS3 | flexbox | flex-wrap

# CSS3 | Layout | flexbox | flex-wrap

Wünschen Sie sich ein Verhalten wie bei float, wo die Elemente in die nächste Reihe umbrechen, dann bietet Ihnen das Flexbox-Modell auch hier eine CSS-Eigenschaft mit **flex-wrap** an.
Der **Standardwert nowrap** verhindert, dass die Elemente in der Flexbox umbrochen werden. Verwenden Sie hierfür den Wert **wrap**, brechen auch bei den Flexboxen die Elemente in eine neue Reihe um, wie Sie es von float her kennen.

Neben nowrap und wrap für die CSS Eigenschaft flex-wrap gibt es den Wert wrap-reverse, mit dem Sie flexible Elemente nach oben umbrechen.




```
section{
    width: 70%;
    height: auto;
    border: 2px solid black;
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row;/*Standardwert*/
    flex-wrap: wrap;
```

```
flex-wrap: wrap;
```

```
article{
    border: 2px solid blue;
    padding: 0.5rem;
    width: 250px;
    height: auto;
}
```

# CSS3 | flexbox | justify-content

# CSS3 | Layout | flexbox | justify-content

Wenn innerhalb eines Flex-Containers nach der Verteilung der Flex-Items noch Leerraum vorhanden ist, kann man diesen mit der **Eigenschaft justify-content auf der Hauptachse verteilen**.
Das Verb to justify heißt auf Deutsch so viel wie bündig machen, **justify-content dient zum Ausrichten von Flex-Items.** Der **Standardwert ist justify-content: flex-start**, auf Deutsch etwa: Richte die Flex- Items am Anfang der Hauptachse aus.

```
display: flex;
flex-direction: row; /*Standardwert*/
justify-content: flex-start;
```

Optisch sieht flex-start aus wie linksbündig, aber nach dem Drehen der Hauptachse kann flex-start auch oben oder rechts sein.

**Das Flexbox-Modell ist richtungsunabhängig, und deshalb werden keine richtungsorientierten Begriffe wie horizontal, vertikal, links, rechts, oben oder unten verwendet.**

# CSS3 | Layout | flexbox | justify-content

Die einzelnen Elemente können Sie mit der CSS-Eigenschaft **justify-content** anordnen.
Mögliche Werte hierfür sind:

- **flex-start,**
- center,
- space-between
- space-around
- space-evenly

```
section{
    width: 70%;
    height: auto;
    border: 2px solid bl
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row; /*Standardwert*/
    justify-content: flex-start;
```

**Artikel 1**
Lorem ipsum dolor sit amet, consectetur adipisicing elit. Reprehenderit, ea a quisquam, impedit libero pariatur amet veniam eaque est facilis aspernatur maiores repudiandae beatae! Doloribus magni obcaecati, eveniet nisi. Inventore dolorum ea quia veniam sapiente odio nam aliquid, labore ex, quis soluta debitis ad, excepturi est magnam magni eaque, iure.

**Artikel 2**
Lorem ipsum dolor sit amet, consectetur adipisicing elit. Maiores fugit et saepe recusandae aliquid minima dolores nisi ratione beatae temporibus explicabo nobis, asperiores soluta sapiente. Cumque porro iure unde accusantium magni, at laboriosam perspiciatis alias in nostrum iste laudantium soluta voluptates! Animi sed itaque aut tenetur, illo quas minima molestias.

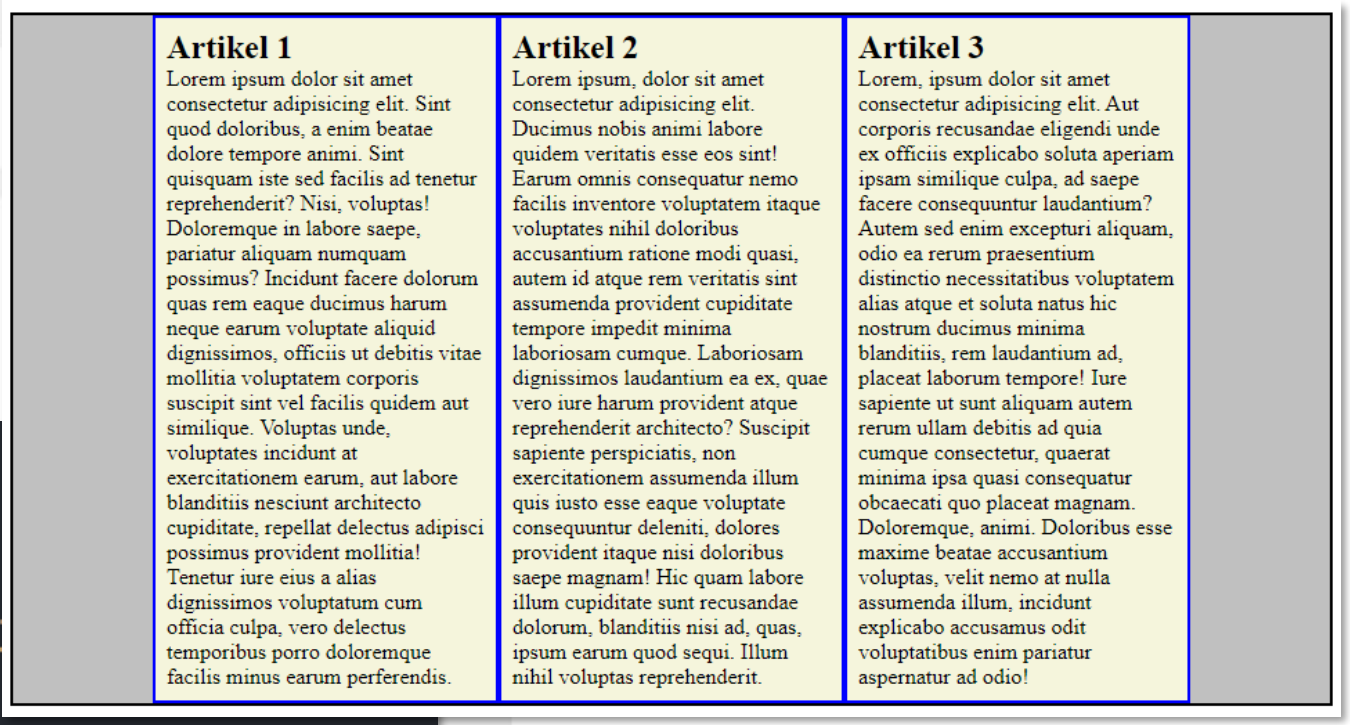
**Artikel 3**
Lorem ipsum dolor sit amet, consectetur adipisicing elit. Repellat possimus dignissimos libero necessitatibus nisi debitis consequuntur saepe doloremque sint dolorum pariatur cumque, atque ratione inventore suscipit, quia, ad aliquam hic sequi! Nisi repellat velit quo corporis quam vel quaerat natus consequatur aliquam cumque deserunt officia, culpa, eius expedita cum maiores?

# CSS3 | Layout | flexbox | justify-content

Die einzelnen Elemente können Sie mit der CSS-Eigenschaft **justify-content** anordnen.
Mögliche Werte hierfür sind:

- flex-start,
- **center,**
- space-between
- space-around
- space-evenly

```
section{
    width: 70%;
    height: auto;
    border: 2px solid b
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row; /*Standa ert*/
    justify-content: center;
```



**Artikel 1**
Lorem ipsum dolor sit amet consectetur adipisicing elit. Sint quod doloribus, a enim beatae dolore tempore animi. Sint quisquam iste sed facilis ad tenetur reprehenderit? Nisi, voluptas! Doloremque in labore saepe, pariatur aliquam numquam possimus? Incidunt facere dolorum quas rem eaque ducimus harum neque earum voluptate aliquid dignissimos, officiis ut debitis vitae mollitia voluptatem corporis suscipit sint vel facilis quidem aut similique. Voluptas unde, voluptates incidunt at exercitationem earum, aut labore blanditiis nesciunt architecto cupiditate, repellat delectus adipisci possimus provident mollitia! Tenetur iure eius a alias dignissimos voluptatum cum officia culpa, vero delectus temporibus porro doloremque facilis minus earum perferendis.

**Artikel 2**
Lorem ipsum, dolor sit amet consectetur adipisicing elit. Ducimus nobis animi labore quidem veritatis esse eos sint! Earum omnis consequatur nemo facilis inventore voluptatem itaque voluptates nihil doloribus accusantium ratione modi quasi, autem id atque rem veritatis sint assumenda provident cupiditate tempore impedit minima laboriosam cumque. Laboriosam dignissimos laudantium ea ex, quae vero iure harum provident atque reprehenderit architecto? Suscipit sapiente perspiciatis, non exercitationem assumenda illum quis iusto esse eaque voluptate consequuntur deleniti, dolores provident itaque nisi doloribus saepe magnam! Hic quam labore illum cupiditate sunt recusandae dolorum, blanditiis nisi ad, quas, ipsum earum quod sequi. Illum nihil voluptas reprehenderit.

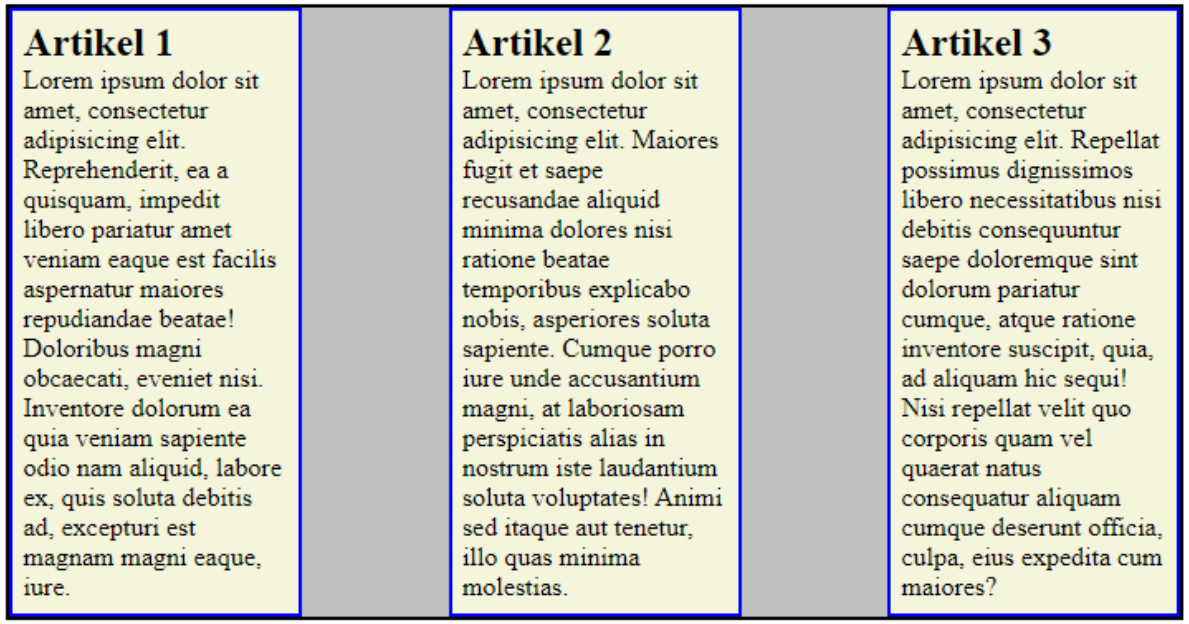
**Artikel 3**
Lorem, ipsum dolor sit amet consectetur adipisicing elit. Aut corporis recusandae eligendi unde ex officiis explicabo soluta aperiam ipsam similique culpa, ad saepe facere consequuntur laudantium? Autem sed enim excepturi aliquam, odio ea rerum praesentium distinctio necessitatibus voluptatem alias atque et soluta natus hic nostrum ducimus minima blanditiis, rem laudantium ad, placeat laborum tempore! Iure sapiente ut sunt aliquam autem rerum ullam debitis ad quia cumque consectetur, quaerat minima ipsa quasi consequatur obcaecati quo placeat magnam. Doloremque, animi. Doloribus esse maxime beatae accusantium voluptas, velit nemo at nulla assumenda illum, incidunt explicabo accusamus odit voluptatibus enim pariatur aspernatur ad odio!

# CSS3 | Layout | flexbox | justify-content

Die einzelnen Elemente können Sie mit der CSS-Eigenschaft **justify-content** anordnen.
Mögliche Werte hierfür sind:

- flex-start,
- center,
- **space-between**
- space-around
- space-evenly

```
section{
    width: 70%;
    height: auto;
    border: 2px solid black;
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row; /*Standardwert*/
    justify-content: space-between;
```



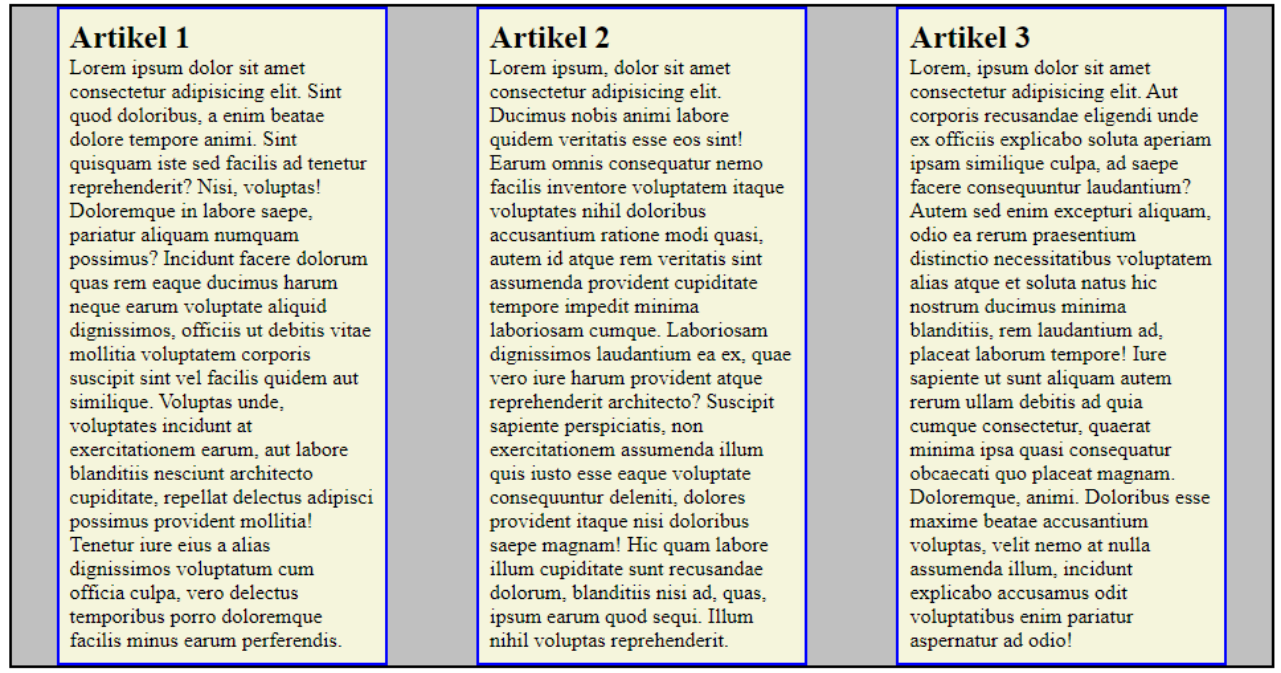

**space-between verteilt den Leerraum gleichmäßig zwischen den Flex-Items**. Am Anfang und am Ende ist kein Leerraum.

# CSS3 | Layout | flexbox | justify-content

Die einzelnen Elemente können Sie mit der CSS-Eigenschaft **justify-content** anordnen.
Mögliche Werte hierfür sind:

- flex-start,
- center,
- space-between
- **space-around**
- space-evenly

```
section{
    width: 70%;
    height: auto;
    border: 2px solid bl
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row; /*Stan wert*/
    justify-content: space-around;
```



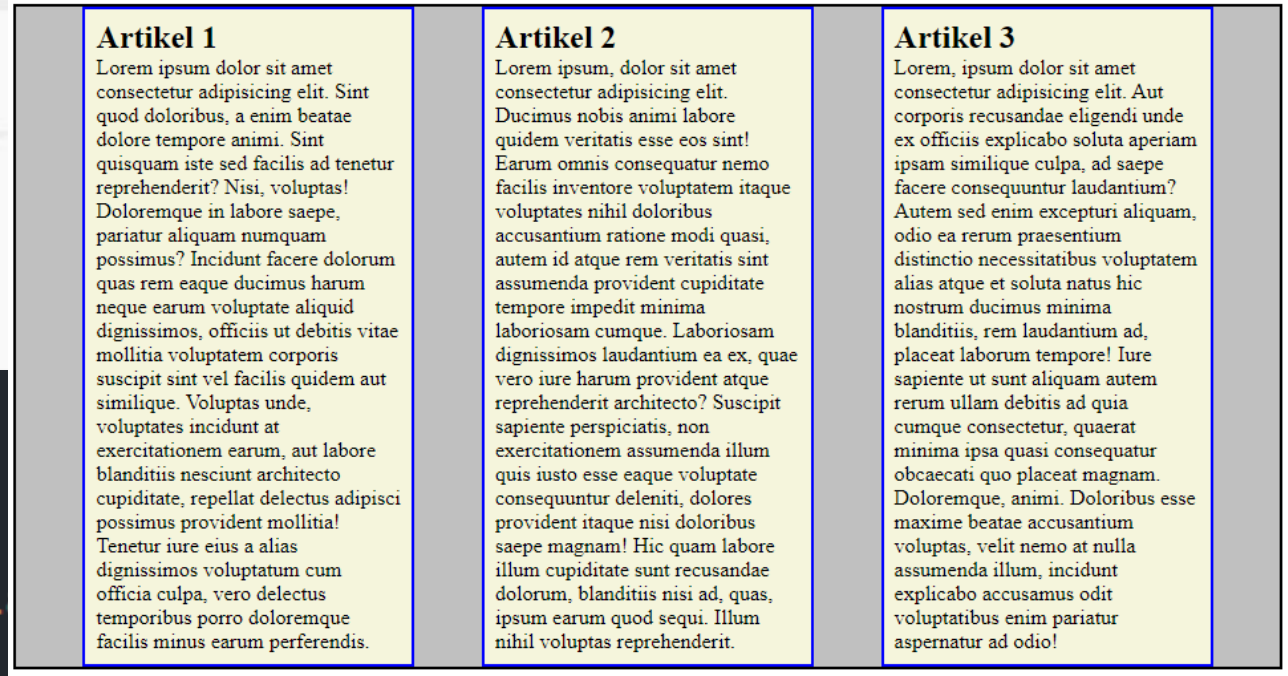

**space-around verteilt den Leerraum gleichmäßig um jedes Flex-Item.**
Zwischen den Flex-Items ist doppelt so viel Platz wie am Anfang und Ende.

# CSS3 | Layout | flexbox | justify-content

Die einzelnen Elemente können Sie mit der CSS-Eigenschaft **justify-content** anordnen.
Mögliche Werte hierfür sind:

- flex-start,
- center,
- space-between
- space-around
- **space-evenly**

```
section{
    width: 70%;
    height: auto;
    border: 2px solid bl
    background: silver;
    margin: 2rem auto;
    display: flex;
    flex-direction: row; /*andardwert*/
    justify-content: space-evenly;
```



**Artikel 1**
Lorem ipsum dolor sit amet consectetur adipisicing elit. Sint quod doloribus, a enim beatae dolore tempore animi. Sint quisquam iste sed facilis ad tenetur reprehenderit? Nisi, voluptas! Doloremque in labore saepe, pariatur aliquam numquam possimus? Incidunt facere dolorum quas rem eaque ducimus harum neque earum voluptate aliquid dignissimos, officiis ut debitis vitae mollitia voluptatem corporis suscipit sint vel facilis quidem aut similique. Voluptas unde, voluptates incidunt at exercitationem earum, aut labore blanditiis nesciunt architecto cupiditate, repellat delectus adipisci possimus provident mollitia! Tenetur iure eius a alias dignissimos voluptatum cum officia culpa, vero delectus temporibus porro doloremque facilis minus earum perferendis.

**Artikel 2**
Lorem ipsum, dolor sit amet consectetur adipisicing elit. Ducimus nobis animi labore quidem veritatis esse eos sint! Earum omnis consequatur nemo facilis inventore voluptatem itaque voluptates nihil doloribus accusantium ratione modi quasi, autem id atque rem veritatis sint assumenda provident cupiditate tempore impedit minima laboriosam cumque. Laboriosam dignissimos laudantium ea ex, quae vero iure harum provident atque reprehenderit architecto? Suscipit sapiente perspiciatis, non exercitationem assumenda illum quis iusto esse eaque voluptate consequuntur deleniti, dolores provident itaque nisi doloribus saepe magnam! Hic quam labore illum cupiditate sunt recusandae dolorum, blanditiis nisi ad, quas, ipsum earum quod sequi. Illum nihil voluptas reprehenderit.

**Artikel 3**
Lorem, ipsum dolor sit amet consectetur adipisicing elit. Aut corporis recusandae eligendi unde ex officiis explicabo soluta aperiam ipsam similique culpa, ad saepe facere consequuntur laudantium? Autem sed enim excepturi aliquam, odio ea rerum praesentium distinctio necessitatibus voluptatem alias atque et soluta natus hic nostrum ducimus minima blanditiis, rem laudantium ad, placeat laborum tempore! Iure sapiente ut sunt aliquam autem rerum ullam debitis ad quia cumque consectetur, quaerat minima ipsa quasi consequatur obcaecati quo placeat magnam. Doloremque, animi. Doloribus esse maxime beatae accusantium voluptas, velit nemo at nulla assumenda illum, incidunt explicabo accusamus odit voluptatibus enim pariatur aspernatur ad odio!

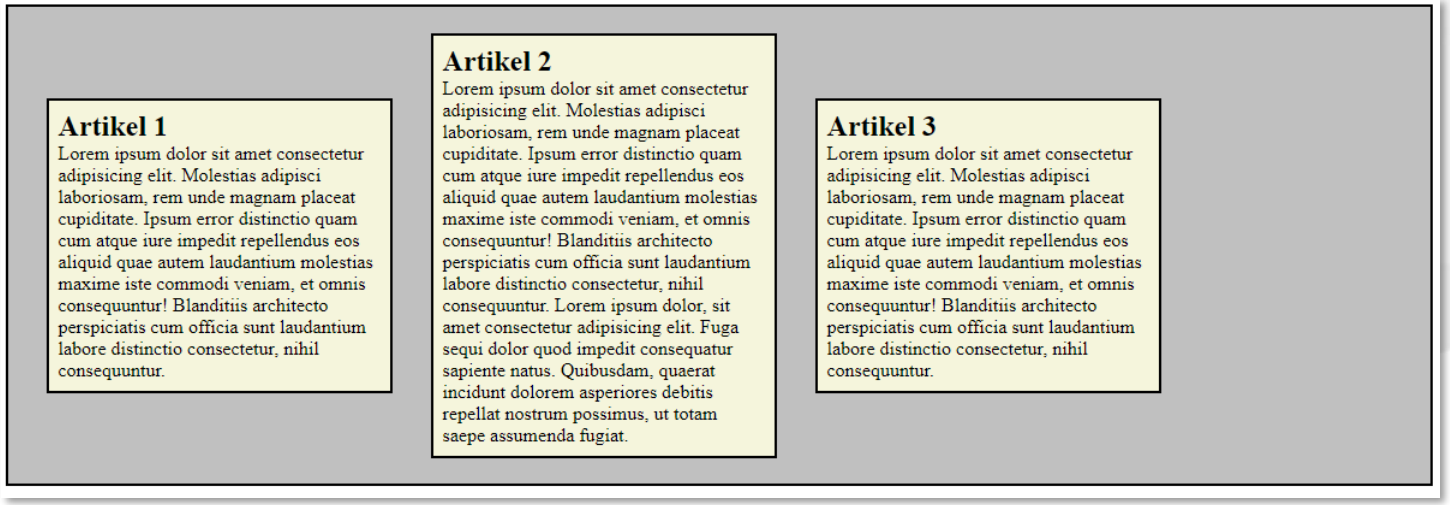
**space-evenly verteilt den Leerraum so, dass vor, zwischen und nach den Flex-Items jeweils gleich viel Platz ist.**

# CSS3 | flexbox | align-items

# CSS3 | Layout | flexbox | align-items

## Ausrichtung entlang der Querachse mit align-items

**Die Eigenschaft align-items verteilt den Leerraum entlang der Querachse und dient zur vertikalen Ausrichtung von Flex-Items. Der Standardwert ist align-items: stretch,** der dafür sorgt, dass die Flex-Items so hoch werden wie der umgebende Flex-Container. **Außerdem gibt es noch die Werte flex-start, center, flex-end und baseline.** Falls Sie also einmal keine gleich hohen Flex-Items möchten, ist align-items die Eigenschaft der Wahl.




```css
section {
  width: 80%;
  height: auto;
  border: 2px solid black;
  background-color: silver;
  padding: 1rem;
  margin: 2rem auto;
  display: flex;
  /* align-items: flex-start; */
  /* align-items: flex-end; */
  align-items: center;
  /* align-items: baseline; */
}
article {
  background-color: beige;
  border: 2px solid black;
  padding: 0.5rem;
  margin: 1rem;
  width: 25%;
}
```

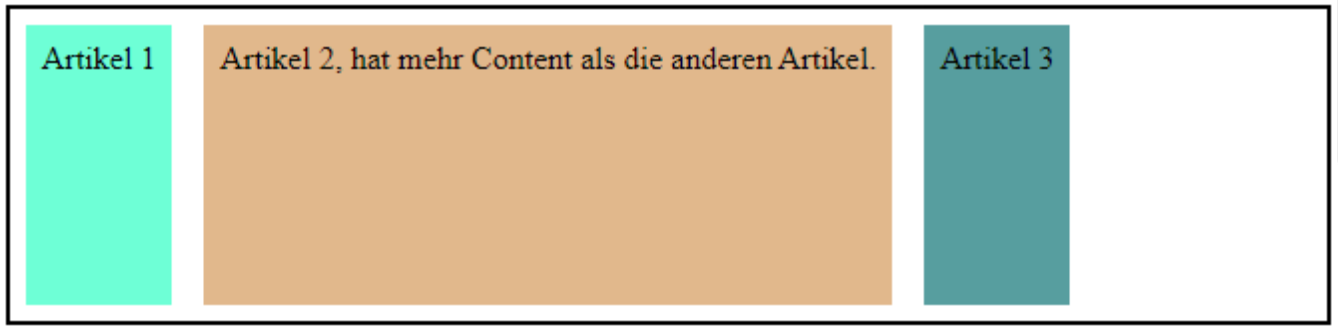
# CSS3 | flexbox | Schlüsselwörter | flex | order

# CSS3 | Layout | flexbox | flex

**Ausdehnung der Flex-Items bestimmen: flex:1 (noch mal gaaaanz einfach)**

Jetzt sehen Sie, wie Sie die Ausdehnung der Flex-Items mit der Eigenschaft flex steuern können.
Die Breite der drei Artikel wird momentan durch deren Inhalt bestimmt.

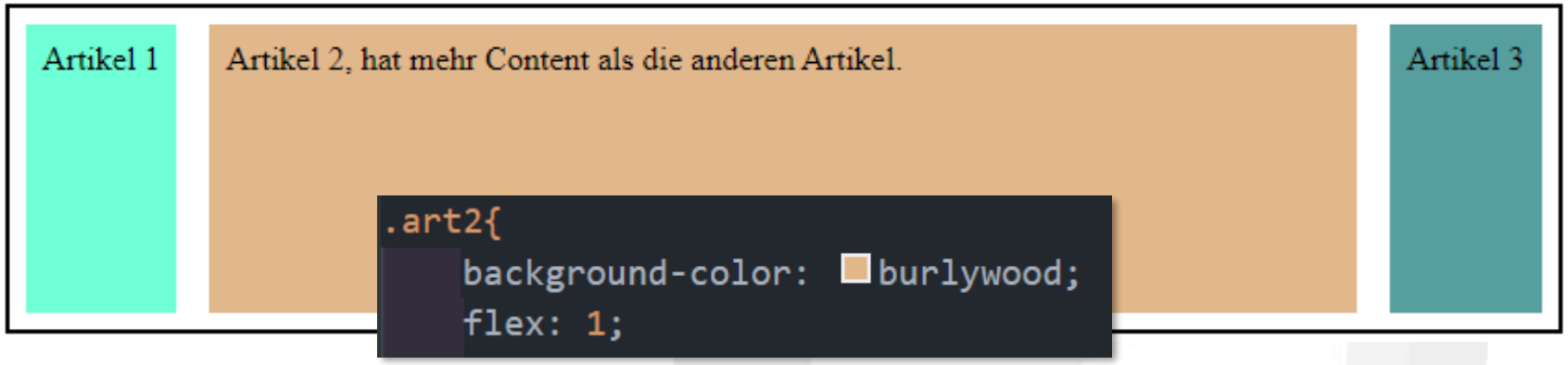
Wenn wir z.B. dem zweiten Artikel ein bisschen mehr Text geben, wird dieser einfach entsprechend breiter.



# CSS3 | Layout | flexbox | flex

**Ausdehnung der Flex-Items bestimmen: flex:1**

Um die Ausdehnung der Flex-Items besser kontrollieren zu können, gibt es eine Eigenschaft namens flex.
Im Folgenden bekommt der zweite Artikel die Anweisung flex:1;

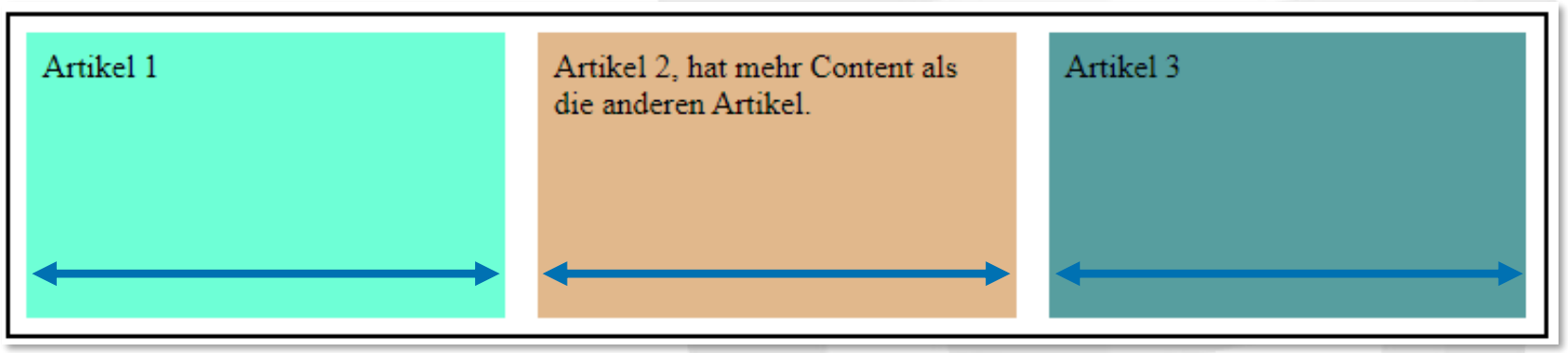



Das Ergebnis zeigt, dass der Artikel mit flex: 1; wächst und den zur Verfügung stehenden Platz vollständig für sich vereinnahmt. Die Breite der beiden äußeren Artikel ist unverändert und wird nach wie vor von deren Inhalt bestimmt.
Der mittlere Artikel hingegen ist gewachsen und drückt den dritten Artikel ganz nach rechts.
**Die Anweisung flex: 1; heißt also so viel wie, „Wachse so lange, bis kein Platz mehr da ist“.**

# CSS3 | Layout | flexbox | flex

**Flex-Items gleichmäßig ausrichten mit dem Schlüsselwort flex: …**

Im folgenden Listing geben Sie nicht nur dem mittleren, sondern allen drei **Artikeln** die Anweisung flex:1:




```
article{
    padding: 0.5rem;
    margin: 0.5rem;
    flex: 1;
    /* flex: auto; */
}
```

Das Ergebnis ist, dass die drei Artikel mit flex:1; gleichmäßig im Flex-Container verteilt werden. Der etwas längere Text in Box 2 bricht jetzt um, da die Breite der Flex-Items nicht mehr durch den Inhalt bestimmt wird, sondern durch flex:1.

**Die Anweisung flex:1; bewirkt, dass das betreffende Element den zur Verfügung stehenden Platz in Anspruch nimmt. Der Platz wird gleichmäßig verteilt.**

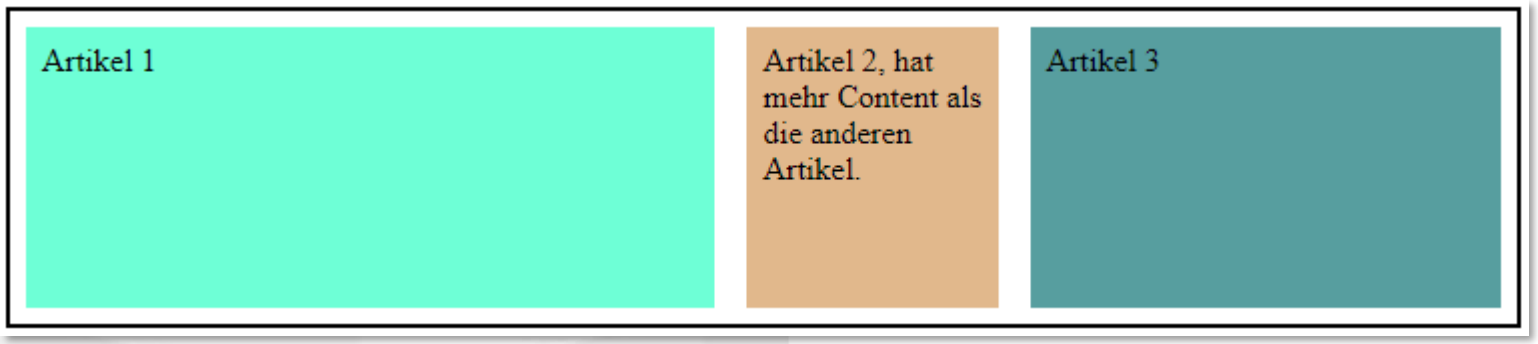
# CSS3 | Layout | flexbox | flex

**Gaaanz einfach: Flex-Items mit unterschiedlicher Breite**

Die Flex-Items sind momentan aufgrund der Anweisung flex:1 alle gleich breit. Die Zahl 1 ist dabei aber keine Größeneinheit, sondern eine Verhältnisangabe, die die zur Verfügung stehende Fläche aufteilt:

**Wenn alle Artikel flex:1 haben, sind alle gleich breit. Wenn ein Artikel flex:2 bekommt, wird er doppelt so breit.**

```
.art1{
    background-color: aquamarine;
    flex: 3;
    /* order:  */
}
.art2{
    background-color: burlywood;
    flex: 1;
    /* order:  */
}
.art3{
    background-color: cadetblue;
    flex: 2;
    /* order:  */
}
```




**Die Zahlen geben also das Verhältnis der Flex-Items zueinander an,** aber Sie müssen dabei nicht immer mit 1 beginnen. Wenn Sie z.B. gerne mit einem 12-Spalten-Grid arbeiten, ersetzen Sie 3 – 1 – 2 einfach durch 3 – 6 – 3:

# CSS3 | Layout | flexbox | order

**Flex-Items gestalten und Reihenfolge ändern mit dem Schlüsselwort order**

In dreispaltigen Floatlayouts ist es mit viel Mühe und negativen Margins möglich, die Spalten am Bildschirm in einer fast beliebigen Reihenfolge darzustellen. Mit Flexbox ist das buchstäblich so einfach wie 1, 2, 3.

**Die Eigenschaft order, gefolgt von einer positiven oder negativen ganzen Zahl, sagt den Flex-Items, in welcher Reihenfolge sie am Bildschirm erscheinen sollen:**

```
.art1{
    background-color: aquamarine;
    /* flex: 3; */
    order: 3;
}
.art2{
    background-color: burlywood;
    /* flex: 1; */
    order: 1;
}
.art3{
    background-color: cadetblue;
    /* flex: 2; */
    order: 2;
}
```

# CSS3 | Layout | flexbox

**Diplay: flex; ist eine Kurzschreibweise**

Die Eigenschaft flex ist sehr praktisch und wird im Alltag häufig eingesetzt, ist aber genau genommen eine Kurzschreibweise für drei Flexbox-Eigenschaften, die sich allesamt um den Platzbedarf der Flex-Items kümmern.

**Ausgeschrieben bedeutet flex 1: Folgendes:**

- **flex-grow: 1;**
- **flex-shrink: 1;**
- **flex-basis: auto;**

Momentan ist nur die erste Eigenschaft flex-grow wichtig. Sie definiert, wie Flex-Items sich ausdehnen, wenn Platz vorhanden ist. Mit dem Standardwert flex-grow: 0 werden Flex-Items so breit wie ihr Inhalt.
Eine Zahl größer als 0 gibt wie weiter oben gezeigt das Verhältnis der Aufteilung an.

# CSS3 | flexbox | horzontal & vertikal ausrichten

# CSS3 | Layout | flexbox

**Elemente horizontal und vertikal zentrieren: margin: auto;**
Die gute alte Eigenschaft margin bekommt **innerhalb** einer Flexbox quasi Superkräfte, da der Wert auto sich jetzt endlich so auswirkt,
wie die meisten CSS-Einsteiger es erwarten: **margin: auto; zentriert ein Element innerhalb einer Flexbox, und zwar nicht nur horizontal, sondern auch vertikal.** Die vertikale Zentrierung eines Elements gelingt mit herkömmlichem CSS nur mit diversen Tricks, innerhalb einer Flexbox ist es gaaaanz einfach.

```html
<div class="aussen">
    <div class="innen">
        <p>margin: auto; <br>testen</p>
    </div>
</div> <!-- class aussen schließt -->
```

```css
.aussen{
    background: lemonchiffon;
    width: 30vw;
    height: 50vh;
    display: flex;
    margin: 3rem auto;
}
.innen{
    font-size: 1.5rem;
    font-family: sans-serif;
    font-weight: bold;
    background: lightgray;
    border: 4px solid lightcoral;
    padding: 1rem;
    margin: auto;
    text-align: center;
}
```

# CSS3 | Layout | flexbox

**Elemente horizontal und vertikal zentrieren: margin: auto;**

```html
<body>
    <div class="flexcontainer">
        <img src="budweisser.png" alt="Verschluss">
    </div>
</body>
```

**Das klappt natürlich auch mit Bildern!**

```style
<style>
    * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
    }

    .flexcontainer {
        width: 60vw;
        height: 70vh;
        background-color: beige;
        border: 2px solid black;
        margin: 3rem auto;
        display: flex;
    }

    img {
        display: block;
        margin: auto;
    }
</style>
```

# CSS3 | Layout | flexbox

**Noch mehr Flexbox:**

https://www.mediaevent.de/css/display-flex.html

https://css-tricks.com/snippets/css/a-guide-to-flexbox/

https://wiki.selfhtml.org/wiki/CSS/Tutorials/Flexbox/Flex-Container

https://codepen.io/justd/full/yydezN/

# CSS3 | Layout | flexbox | VSC Extension

**Es gibt für Visual Studio Code eine Extension, welche vielleicht helfen kann.**

VS-Code-Erweiterung, mit der Sie ein Flexbox-Cheatsheet direkt im Editor  öffnen können.
Merkmale: Das Cheatsheet kann auf verschiedene Arten geöffnet werden. Drücken Sie Strg + Umschalt + P „Open CSS Flexbox Cheatsheet“.

**Drücken Sie die Tastenkombination Strg + KK**. Bewegen Sie den Mauszeiger über eine beliebige display: flex-Deklaration und klicken Sie im Popup auf den Link „CSS Flexbox Cheatsheet öffnen“.

Neben dem Link „Open CSS Flexbox Cheatsheet“ im Hover-Popup wird auch ein Bild für jede Flexbox-Eigenschaft angezeigt.

Dies soll es einfacher machen, die Funktionsweise der einzelnen Eigenschaften zu verstehen, ohne das Spickzettel öffnen zu müssen.

# VIELEN DANK!